

1 Pattern Matching

A new syntactic category: *patterns* $\mathcal{P}$:

$\mathcal{P}$	=	$\mathbb{N}$	— number
		$-$	— wildcard
		$[\mathcal{P}^*]$	— array
		$\mathcal{E} \mathcal{P}^*$	— S-expression
		$\mathcal{X} @ \mathcal{P}$	— named pattern

Concrete syntax mainly repeats the abstract; in S-expression patterns extra round brackets are used to delimit the constructor's name from arguments (if any); arguments of array/S-expression patterns are delimited by commas, and extra round brackets can be used to group subpatterns. Additionally, one derived form is used: an identifier x is treated as a pattern $x@_$.

Pattern matching expression:

$$\mathcal{E}^+ = \text{case } \mathcal{E} \text{ of } (\mathcal{P} \times \mathcal{E})^+ \text{ esac}$$

In a concrete syntax branches of case expression are delimited by “—”, and in each branch “ $\rightarrow$ ” is used to delimit pattern from expression.

Well-formedness of case expressions is established in an obvious manner:

$$\frac{e : \text{Val} \quad e_i : a}{\text{case } e \text{ of } p_1 \rightarrow e_1 \dots p_k \rightarrow e_k \text{ esac} : a}$$

2 Operational Semantics

There are two aspects that have to be covered in semantic description of pattern matching:

- the criterion for a *scrutinee* to be matched by a pattern;
- the discipline of binding support.

The latter aspect is covered by a desugaring. First, we define a mapping

$$\beta : \mathcal{P} \rightarrow \mathcal{E} \rightarrow \mathcal{X} \rightarrow \mathcal{E}$$

in a following manner:

$$\begin{aligned} \beta n e &= \lambda _ . \perp \\ \beta _ e &= \lambda _ . \perp \\ \beta ([p_0 \dots p_k]) e &= \\ \beta (C p_0 \dots p_k) e &= (\beta p_0 e[0]) [\beta p_1 e[1]] \dots [\beta p_k e[k]] \\ \beta (x @ p) e &= (\beta p e)[x \leftarrow e] \end{aligned}$$

This function determines a proper *subvalue* of an expression bound by an identifier in a pattern. For example, for a pattern $[x, C(_, y)]$ and scrutinee s the value of $\beta[x, C(_, y)] s$ can be described by the following table:

$$\begin{aligned} x &\rightarrow s[0] \\ y &\rightarrow s[1][1] \end{aligned}$$

Then, given a pattern-matching expression **case** e **of** ... we, first, bind the expression e to a *fresh* variable, say, s :

var $s = e$;
case s **of** ...

Then, we transform each branch $p \rightarrow e$ into the following:

$$\begin{aligned} p &\rightarrow \mathbf{var} \ b_1 = \beta p s b_1, \\ &\quad \dots \\ &\quad b_k = \beta p s b_k; \\ &\quad e \end{aligned}$$

where $b_1, \dots, b_k$ are all bindings in p .

Now, for determining the descipline of matching we need an extra relation

$$match \subseteq \mathcal{P} \times \mathcal{V}$$

between patterns and values. We define it in a following way:

$$\begin{aligned} &match(-, v) \\ &match(n, n) \\ &\frac{match(p_i, v_i)}{match([p_1, \dots, p_k], [v_1, \dots, v_k])} \\ &\frac{match(p_i, v_i)}{match(C p_1, \dots, p_k, C v_1, \dots, v_k)} \\ &\frac{match(p, v)}{match(x@p, v)} \end{aligned}$$

Finally, the operational semantics of pattern-matching expression can be given by the following rules:

$$\begin{aligned} &c \xRightarrow{e} \langle c', v \rangle \\ &\frac{v \vdash c' \xrightarrow{(p_1, e_1) \dots (p_k, e_k)}_{\mathcal{P}} c''}{c \xRightarrow{\mathbf{case} \ e \ \mathbf{of} \ p_1 \rightarrow e_1 \ \dots \ p_k \rightarrow e_k \ \mathbf{esac}} c''} \end{aligned}$$

where an additional transition “ $\rightarrow_{\mathcal{P}}$ ” is defined as follows:

$$\frac{\begin{array}{c} match(p, v) \\ c \xRightarrow{e} c' \end{array}}{v \vdash c \xrightarrow{(p, e)ps} \wp c'}$$

$$\frac{\begin{array}{c} \neg match(p, v) \\ v \vdash c \xrightarrow{ps} \wp c' \end{array}}{v \vdash c \xrightarrow{(p, e)ps} \wp c'}$$